Multi-Output Transactions

A primitive for Bitcoin micropayments

Henry Hudson · Henceforth · v1.0 / 2026

Companion paper. This document accompanies the main Henceforth book (hforth.pdf) and assumes familiarity with the Bitcoin white paper, the BSV UTXO model, and the FORTH terminal. It documents the design and implementation of two terminal words — send and send-many — and the case for treating multi-output transactions as the default unit of payment rather than the exceptional one.

Based on findings from Craig Wright (aka Satoshi Nakamoto). The work documented here implements components of the system described in Craig Wright, Batching, Headers, and Throughput: Operating a Bitcoin SV Wallet with Offline Synchronisation and High-Volume Microtransactions, Craig's Substack, 2026-05-02. The article frames a wallet capable of high-volume operation as built on three pillars — header-based blockchain awareness (SPV), batch construction of transactions, and structured UTXO management for both sending and receiving. Henceforth had the first and most of the third before this paper was written. The send and send-many words documented here close the second. Section 7 maps the full architecture against the current implementation, naming the gaps explicitly.

1 Background

Section 7 of the Bitcoin white paper, Combining and Splitting Value, contains a sentence that has shaped wallet design for fifteen years:

> "To allow value to be split and combined, transactions contain multiple inputs and outputs. Normally there will be either a single input from a larger previous transaction or multiple inputs combining smaller amounts, and **at most two outputs**: one for the payment, and one returning the change, if any, back to the sender."

At most two. The phrasing is conservative — in 2008 it described the dominant case — but it was never a protocol limit. Bitcoin transactions accept an arbitrary number of outputs subject only to the standard maximum transaction size. The protocol primitive

is **a single transaction with N outputs**. The two-output convention is a custom that wallets and exchanges have collectively settled on, mostly out of inertia.

If you accept that the conservative case is a default rather than a ceiling, a great deal becomes possible.

This paper is about one of those possibilities: paying a list of recipients atomically in a single signed transaction. The technique is structurally identical to the standard send — same UTXO selection, same signing path, same broadcast — and adds one capability that single-output sends cannot match: N payments at constant input cost. For micropayment workflows, that constant matters more than any other property of the transaction.

Wright's 2026 article Batching, Headers, and Throughput makes the same point and presses further: it argues that for any wallet expected to operate at the receive side of micropayment flows — where UTXO sets accumulate by the hundreds — batching is not a fee optimisation but a structural requirement. A wallet that does not batch its outgoing payments will, over time, also fail to consolidate its incoming ones; the two problems compound. The work documented in this paper implements batching on the send side; the consolidation routine (ConsolidateUTXO in the wallet code, surfaced as the Consolidate action in the UTXO detail view) closes the receive side. The two together form the structured UTXO management Wright calls foundational.

2 The Mathematics of Overhead

A serialized BSV transaction has three components, each with a known byte cost. Henceforth carries these as named constants in ExtensionsForNumberFormatters.swift so no part of the codebase fee-estimates with raw literals:

Transaction byte sizes

```
p2pkhInputSize   = 148    // input + signature + pubkey + index
p2pkhOutputSize  = 34     // value + script-length + P2PKH script
txOverheadSize   = 10     // version + input count + output count
                          // + locktime
```

The total wire size of a 1-input, N-output transaction is therefore:

$$\text{size}(N) = 148 + 34N + 34_{\text{change}} + 10 = 192 + 34N$$

(One input, N payment outputs, one change output back to the sender, plus the per-transaction header.)

The same total amount paid across N separate single-payment transactions costs:

$$\text{size}_{\text{single}}(N) = N \cdot (148 + 34 + 34 + 10) = 226N$$

The ratio is what makes batching worth talking about. At $N = 8$ recipients:

	Single tx, 8 outputs	Eight separate txs	Ratio
Wire bytes	464	1808	0.26×
Fee at 0.05 sat/byte	24	91	0.26×
ARC submissions	1	8	0.125×
Biometric prompts (terminal)	1	8	0.125×
Mempool entries	1	8	0.125×
Atomicity	all-or-nothing	per-payment	—

For payments dominated by economically large transfers — rent, mortgage, salary — a 67-satoshi fee difference is irrelevant. For micropayments, where individual amounts may themselves be in the low thousands of satoshis, paying four times the fee to avoid batching is the difference between economical and uneconomical workflows. A 14,000-sat coffee subscription payment paying an 11-sat fee retains 99.92% of its value to the recipient. Sending eight of those as eight transactions imposes 88 sats of fee on 112,000 sats of value — still small, but no longer trivially small, and the cost grows linearly with the number of recipients you treat as independent.

> **The white paper anticipates this.** Section 7 says "at most two outputs" but the same paragraph notes that transactions "contain multiple inputs and outputs." The protocol primitive is N-to-M; the two-output case is a convention. Multi-output sending is not an extension to Bitcoin — it is the unconstrained form of what every wallet already does.

3 The Henceforth Implementation

Henceforth exposes two terminal words for sending value:

Send words

```
send        (amount c-addr u -- )
            Pay `amount` satoshis to the address (or
            space-separated addresses) specified by the
            counted string at (c-addr, u). When the string
            contains multiple addresses, the amount is split
            equally across them.

send-many   (amount1 ... amountN c-addr u -- )
            Pay individual amounts to N addresses. N is
            derived from the address string at the top of
            the stack; the word then pops exactly N amounts
            below it. amount[i] pairs with address[i] in the
            order the user pushed them.
```

Both words share one rule: amounts first, recipient string at the end. The string is placed at the top of the stack so the word can read it first, derive N from the space-separated address count, and only then consume the right number of amounts. The alternative we considered — recipient string at the bottom plus an explicit N parameter — carried redundant ceremony, since the address string already declares the

count implicitly. Removing the explicit count parameter makes the contract harder to misuse.

3.1 Stack discipline

A successful invocation of send-many consumes the entire stack contribution and leaves no residue:

send-many stack trace

```
> 320000 95000 42500 14000 8500 22000 18000 250000
>  ok.   ( 8 amounts on the stack )

> s" 1Land... 1Groc... 1Util... 1Cof... 1News... 1Gym... 1Stream
  ... 1Cold..."
>  ok.   ( S" pushes (c-addr, u) on top; addresses live in data
  memory )

> send-many
send-many: queued 8 outputs totalling 770000 sats
 ok.   ( all 10 stack items consumed atomically )
```

If any validation step fails — one bad address, one zero amount, total exceeding wallet balance — the entire stack is restored to its pre-call state. The user can inspect, correct the input, and retry. This is a property of the implementation, not the protocol: the Swift function pops nothing it cannot complete, and on every error path it re-pushes the items in the order the user originally placed them.

3.2 Why this shape and not another

Three alternative designs were tried before settling on the current one:

- **v1** — read the address from terminalResponses.last (the ." channel). Coupled the address parse to terminal output state. Any future change to how terminal lines were buffered would have silently broken sending.
- **v2** — S" with stack effect (c-addr u amount –). Decoupled from terminal output, but reads unnaturally: “the address, then the amount, then send.”
- **v3 (shipped)** — S" with stack effect (amount c-addr u –). Reads as “n satoshis to address x.” Same shape as send-many, which couldn’t be flipped without losing the implicit-N property described above.

The cost of v3 is asymmetry with English (we say “pay alice 100,” not “100 to alice”) — the benefit is the implicit-N contract and a single rule across both words. Consistency between two words the user constantly switches between is worth more than alignment with English word order.

4 Worked Example: One Weekly Bill, One Transaction

Consider a freelancer paying their fixed weekly costs every Friday. Eight recipients, eight separate amounts, all in BSV:

Recipient	Address (truncated)	Amount (sats)
Landlord (weekly rent share)	1Land...kqW	320,000
Groceries (paymail → corner store)	1Groc...xR2	95,000
Electricity & internet	1Util...mB7	42,500
Coffee subscription	1Cof...p3J	14,000
Newspaper (digital, weekly)	1News...t9K	8,500
Gym (off-peak)	1Gym...sV4	22,000
Streaming bundle	1Strm...e6W	18,000
Cold wallet (weekly auto-savings)	1Cold...zY1	250,000
Total		**770,000**

The whole payment is one line at the terminal:

Weekly bill: one transaction

```
> 320000 95000 42500 14000 8500 22000 18000 250000
> s" 1Land...kqW 1Groc...xR2 1Util...mB7 1Cof...p3J
>    1News...t9K 1Gym...sV4 1Strm...e6W 1Cold...zY1"
> send-many

Confirm send-many to 8 recipients (770000 sats)
 [Face ID prompt]

send-many: queued 8 outputs totalling 770000 sats
 ok.
```

4.1 What goes on the wire

The serialized transaction is one 1-input, 9-output structure (eight payment outputs plus one change output back to a freshly-derived change address):

$$\text{size} = 148 + 8 \cdot 34 + 34_{\text{change}} + 10 = 464 \text{ bytes}$$

At a current ARC fee rate of 0.05 sat/byte that is a 24-satoshi fee — 0.003% of the 770,000-sat payment. Eight separate transactions for the same eight payments would be $8 \cdot 226 = 1808$ bytes and approximately 91 sats of fee. The fee saving is small in absolute terms, but compounded weekly — 52 weeks at four extra fees-per-week saved is around 3,500 satoshis per year per workflow — and the structural savings are larger:

- **One biometric prompt.** The user is not asked to authenticate eight times for what is conceptually a single decision.
- **One ARC submission.** One round trip to GorillaPool. If GorillaPool fails, one failover to TAAL — not eight.

- **One mempool entry.** The eight payments are seen by the network together, settled together, and confirmed together. No payment is in the mempool while another has already confirmed.
- **Atomicity.** If the landlord's address contains a typo, no payments are sent. With eight separate transactions, the first seven could broadcast successfully before the eighth fails — leaving the user with a half-paid bill list and no clean retry path.
- **One transaction in history.** The wallet's transaction list shows "Friday bills: 770,000 sats, 8 recipients" instead of eight near-simultaneous outgoing entries that the user has to mentally re-group.

4.2 What goes on-chain

The on-chain shape is identical to any other Bitcoin transaction. From the network's perspective, the freelancer paid 770,000 satoshis from one input to eight payees, with change returned to themselves. Every recipient sees a normal P2PKH output addressed to them, indistinguishable from a single-purpose payment. There is no batching protocol — the protocol primitive is the single transaction, and it always supported N outputs.

The landlord's wallet, polling for new transactions to its receive address, gets a 320,000-sat outpoint exactly as if the freelancer had paid them alone. So does each of the other seven recipients. None of them needs to know they were one item on a list, and the on-chain footprint is a single 464-byte transaction instead of 1,808 bytes spread across eight independent ones.

5 Validation, Approval, and Failure Modes

send-many runs a five-step validation cascade before touching the wallet. Every step either succeeds or restores the stack to its pre-call state with a specific error message — there is no partial state.

1. **Stack arity.** The integer stack must hold at least the (c-addr, u) pair on top. Without those two, the word cannot even read the address string. Failure: Stack underflow {send-many}.
2. **Address string non-empty and parseable.** The string at (c-addr, u) is recovered from data memory and split on whitespace. Empty string or zero-address result: re-push (c-addr, u) and return Invalid address string or No addresses parsed.
3. **Amount count matches address count.** With N addresses parsed, the stack must hold at least N more integers below the (c-addr, u) pair. Failure message reports both numbers: "N addresses parsed; need N amounts on the stack."
4. **All amounts positive.** Zero or negative amounts are rejected with the full set restored in original push order. This is a hard invariant: BSV outputs of zero satoshis are technically legal but always wrong intent, and dust thresholds make small-positive outputs unhelpful.

5. **Each address structurally valid.** Wallet.validateBSVAddress (Base58Check + version byte + checksum) is called on every address. The first failure aborts with the offending address quoted.

After validation, the total amount is compared against the wallet's confirmed + unconfirmed balance — a fast local check that fails before the biometric prompt, so the user is not asked to authenticate for a transaction that cannot build.

5.1 Biometric approval is scoped, not per-output

The approval message reads "Confirm send-many to 8 recipients (770000 sats)." One Face ID prompt, one decision. The 120-second approval cache (PaymentApproval.swift) is keyed on the surface (.terminal) and the total amount in satoshis, so a follow-up send-many with the same total within the window does not re-prompt — which is correct for the typical pattern of running an automation that issues two related batches.

Crucially, the user is approving a labelled batch of N payments totalling X sats, not the bytes of a constructed transaction. The current approval flow shows the total and the recipient count but does not enumerate the inputs being spent. This is an acknowledged limitation of the current implementation, not a property of multi-output sending — the same limitation applies to single-output send. Surfacing input decomposition (which UTXOs are being consumed, from which addresses) is on the post-launch backlog under the BRC-100 createAction approval-flow work.

5.2 Cold Mode

When the active wallet has Cold Mode enabled (WalletCard.coldModeEnabled = true), send-many does not broadcast. Instead, the transaction is built, signed, and queued in the Pending Broadcasts view exactly as a single-output send would be. The user manually broadcasts later from a network-connected device. No part of the multi-output construction — UTXO selection, fee estimation, signing, change derivation — requires the network. Only the final ARC submission does.

6 Why This Belongs in a Terminal

A graphical wallet UI for "pay eight recipients in one transaction" is not impossible — it just requires a custom screen, a custom data entry pattern, and a custom approval flow, all of which are downstream of someone deciding that batched payment is a feature worth UI investment. Henceforth's terminal makes the same operation a one-liner without any of that downstream design work:

Composability

```
: weekly-bills  ( -- )
   320000 95000 42500 14000 8500 22000 18000 250000
   s" 1Land...kqW 1Groc...xR2 1Util...mB7 1Cof...p3J
      1News...t9K 1Gym...sV4 1Strm...e6W 1Cold...zY1"
```

```
    send-many ;

> weekly-bills
Confirm send-many to 8 recipients (770000 sats)
 ok.
```

The user has defined a personal word, weekly-bills, that captures their weekly payment routine in a single token. Running it costs them one biometric tap. Changing an amount or address is a normal edit to the word's definition. Saving the definition in a .fs file makes it portable across devices.

This is the FORTH composability argument applied to payment automation: every word the user defines is a new affordance, and the cost of defining a new affordance is exactly the cost of writing its body once. Multi-output sending is not a feature that has to be added to a UI — it is what falls out of treating the payment primitive as a stack operation.

7 Position within the Wider Architecture

Wright's article treats a high-volume wallet as a system of components, none of which can be removed without loss: offline signer, header sync, watch-only monitoring, batching, consolidation, fresh-address strategy, and a non-negotiable pre-signing verification step. The table below records where Henceforth currently stands against each — explicitly marking what is shipped, what is partial, and what remains.

Component (Wright)	Henceforth implementation	Status
Offline signer (cold storage)	WalletCard.coldModeEnabled, PendingBroadcasts	✓[a]
Header-based blockchain awareness	HeaderSyncService, SPVCheckpoint	✓
Offline header transfer (removable media)	HFHeaderBundle + Air-Gap Tools UI	✓[d]
Watch-only address monitoring	MempoolMonitorService + HFAddressPack push	✓
Merkle-proof export for offline verify	HFReceiptBundle + verifyTransaction(fromBundle:)	✓[d]
Batch construction of transactions	send via PayView; send-many via MultiPaySheet	✓
UTXO consolidation	ConsolidateUTXO + send-time selection drains dust	✓[b]
Fresh address per payment	Type42 address rotation, automatic post-tx	✓
Verification before signing	PayView + MultiPaySheet output review; TransactionReviewView for createAction	✓[c]

[a] Cold Mode is a per-wallet software flag, not a hardware air-gap; the air-gap bundle work below brings the two-machine model within reach.
[b] selectUTXOs switches to a fragmentation-aware policy (raised dust cap, smallest-first phase 2) when the wallet has more than 100 UTXOs — normal sends drain the dust pile as a side effect (Wright §V). Residual polish: a fragmentation banner in PayView's pre-broadcast section so the user sees the consolidation happening, and a transaction-size ceiling fallback for pathologically fragmented wallets (>200 inputs would exceed the standard relay limit).
[c] Every outgoing transaction surface presents the user with the output set before Face ID fires: terminal send pre-fills PayView, terminal send-many presents MultiPaySheet with a per-recipient grid, BRC-100 createAction routes both single and multi-output through the same surfaces. TransactionReviewView is the canonical byte-level review surface (inputs + outputs + fee + privacy hint) wired into the BRC-100 pending-broadcast queue (post-signAction). Hoisting it as an overlay over the active send paths so the input side is reviewable on every transaction is the residual polish here — the surface exists and is tested, it just isn't yet the default on the active path.
[d] Bundle infrastructure (formats, encode/decode, SPVService.verifyTransaction(fromBundle:), Wallet.deriveAddressPack) is in place and unit-tested for round-trip + malformed-rejection. The Settings → Bitcoin Air-Gap Tools UI ships with Export Headers / Export N receive addresses / Import buttons routing by .hf* extension, presenting via .fileExporter / .fileImporter (cross-platform iOS + macOS; AirDrop appears as a destination in the system share sheet on both). Two-device end-to-end testing is the remaining manual verification step.

The previously-unimplemented rows — offline header transfer and Merkle-proof export — are now fully closed end-to-end. HFHeaderBundle carries a contiguous run of headers plus the source checkpoint hash; the receiving signer imports them through the existing commitValidatedHeaderRun path so chain-link + PoW + checkpoint-anchor invariants enforce themselves on import. HFReceiptBundle carries a raw transaction + TSC (BRC-74) Merkle proof + the containing block header; verifyTransaction(fromBundle:) validates inclusion against the signer's local header at the same height without any network call. HFAddressPack pushes pre-derived receive addresses outward from the signer (where the seed lives) to the watcher (which holds no key material) so the watcher can monitor deposits without ever seeing the seed. The Settings Air-Gap Tools section presents all three via system file pickers, with AirDrop reachable from the iOS and macOS share sheets.

8 Discussion

Three observations from shipping this:

The two-output convention is sticky. Every wallet that exposes only a single-recipient pay screen is silently asserting that the two-output case is the unit of payment. For workflows that genuinely are one-to-one, this is fine. For workflows that are not — weekly bills, payroll, micropayment distributions, royalty splits, multi-party settlements — the convention costs real fees and forces users to manually orchestrate what the protocol already supports atomically.

Atomicity is the underrated property. The fee saving is the easy headline, but the practical user-facing benefit is that the eight payments either all happen or none do. The freelancer in our worked example cannot end up with rent paid and groceries unpaid because of an intermittent ARC failure between the second and third submissions —

there is no second submission. ARC accepts the 464-byte transaction or rejects it; either way the user knows the state of all eight payments with one confirmation.

Naming N to derive N is the small interface decision that paid off. The address string declares N implicitly. Requiring the user to also push N as a separate integer would have been redundant ceremony, and any redundancy in a stack-based interface is an opportunity for the two redundant pieces to disagree. Removing the explicit count made the word strictly harder to misuse and strictly easier to read.

9 Further Work

The Wright plan landed in two waves between this paper's first draft and now: the original scope (send + send-many + atomic batched payments) plus the broader Wright architecture (offline header transfer, Merkle-proof export, watch-only address packs, fragmentation-aware UTXO selection, byte-level review on the BRC-100 queue). Below records what shipped after publication and what remains genuinely outstanding.

Shipped since publication

- **Offline header transfer.** Wright's article (§III) describes batch-exporting headers from an online machine to removable media and re-importing them on the air-gapped signer. Now closed: HFHeaderBundle encodes a contiguous header run + the source checkpoint hash, the receiving signer imports through the existing commitValidatedHeaderRun path so chain-link + PoW + checkpoint-anchor invariants enforce themselves on import, and Settings → Bitcoin → Air-Gap Tools surfaces an Export Headers button + an Import button that routes by .hfheaders extension. AirDrop is reachable from the system share sheet on both iOS and macOS.
- **Merkle-proof export for offline verification of received funds.** HFReceiptBundle carries (txid, raw tx hex, block height, block hash, TSC Merkle proof, containing header). SPVService.verifyTransaction(fromBundle:) validates inclusion against the signer's LOCAL header at the same height with no network call — the bundle's own header is a reference, not a trust source. Decode-time consistency checks (proof.target ≡ blockHash, header.hash ≡ blockHash, header.height ≡ blockHeight) surface tampering before the heavier PoW + merkleroot verification runs. v1 uses raw-tx + TSC; a v2 schema bump may upgrade to BEEF (BRC-62) to carry the ancestor chain needed for input-script validity proofs.
- **Automatic consolidation trigger.** Wright §V–XI: fragmentation is inevitable for any wallet on the receive side of micropayments and periodic consolidation is the only sustainable answer. Wallet.selectUTXOs now switches to a fragmentation-aware policy when the wallet has more than 100 UTXOs (raised dust cap, smallest-first phase 2), so normal sends drain the dust pile as a side effect — the user initiates an ordinary send and the wallet consolidates with no extra prompt. Residual polish: a fragmentation banner in PayView's pre-broadcast section so the user sees "you're about to spend 47 small UTXOs — this transaction consolidates them at no extra cost," and a transaction-size ceiling fallback for pathologically fragmented wallets (>200 inputs would exceed the standard relay limit).

- **send / send-many parity.** The original paper documented send-many routing through MultiPaySheet for visual review while send retained the legacy direct-broadcast path after Face ID. Single-recipient send now also routes through PayView with the amount + address pre-filled, so every outgoing transaction surface honours Wright §VIII's pre-signing verification requirement. The test path retains the direct-broadcast path under FORTH.bypassPaymentApprovalForTests so CI keeps exercising the broadcast pipeline without driving a SwiftUI sheet.
- **Approval cache content-hashing.** A 770,000-sat send-many approval no longer auto-covers a 770,000-sat send to a different recipient within the 120-second window. The cache now keys on (surface, amount, outputsDigest) where outputsDigest = SHA256(sorted [(recipient, sats)]), so the cache is bound to the actual recipient set as well as the surface and amount cap. Migration is monotonically safer: call sites that haven't passed real digests yet store empty digests, which never cache-hit non-empty ones — the only failure mode is an extra re-prompt, never a silent over-authorise.

Outstanding

- **Cross-input batching (M-to-K).** The current implementation is 1-input-to-N-outputs (and 1-wallet to-N-outputs in the air-gap path). Combining M wallets funding a shared K outputs — multi-party settlement, payment-channel-style atomic exchanges — extends to M-to-K, which the protocol already supports and the Swift transaction builder doesn't yet expose at the terminal layer. The signed-but-unbroadcast format would naturally reuse Item 3's air-gap plumbing (a .hfpsbt bundle along the lines of BIP-174 PSBT, except wire-compatible with the BRC-100 spends map). Until there is concrete demand — a real-world settlement use case rather than a sketch — this stays a v3 design note rather than a build target.
- **Streaming send-many.** For pay-per-second use cases, a higher-level word could accumulate (recipient, rate) tuples over time and flush them on an interval — one transaction per minute or per session instead of one per second. The send-many primitive is the building block; the scheduler is what would be new. Deferred behind real-world demand.
- **Active-send byte-level review.** TransactionReviewView already exists and is wired into the BRC-100 pending-broadcast queue: the user taps Review on a queued createAction and sees a structured inputs/outputs/fee decomposition before authorising the signed-and-deferred broadcast. The remaining work is hoisting that overlay over the active send paths so the input side is also reviewable when typing send or send-many at the terminal — on the active path the user reviews outputs but not inputs. The surface exists and is tested; the residual is the routing decision.

> The point of all of this is small. We removed a redundant count parameter from a stack-based interface, exposed an existing Swift function as a FORTH word, and wrote a short validation cascade. The behaviour it enables — atomic batched payments at constant input cost — has been latent in the Bitcoin protocol since 2008. The change is not the capability; the change is making the capability one line at a terminal.